

Spring Boot

Scenario-Based Interview Q&A

30 real-world questions with practical, production-minded answers

How to use this guide

Each answer starts with the decision or diagnostic approach an interviewer expects. The bullets give concrete actions, and the interview close gives a concise way to finish your response. Adapt the examples to tools used in your own projects.

Coverage: deployment and configuration, performance, resilience, transactions, API design, and project habits.

Contents

1. Deployment, configuration, and operations

These answers show that you start with evidence, then narrow the failure boundary. Questions: 1, 3, 6, 12, 13, 15, 18, 19, 23, 25, 28, 29

2. Performance, capacity, and resilience

Start with measurements and protect the service from slow or failing dependencies. Questions: 2, 4, 8, 9, 10, 11, 17, 20, 21, 22, 26

3. Spring behavior, transactions, and API design

These questions test whether you understand framework behavior in real deployments. Questions: 5, 7, 14, 16, 24, 27

4. Practical project habits

A strong interview answer is specific about observability, trade-offs, and validation. Questions: 30

Tip: When answering live, lead with the first action you would take, name the signal you would inspect, and describe how you would validate the fix.

1. Deployment, configuration, and operations

These answers show that you start with evidence, then narrow the failure boundary.

1. Your Spring Boot app works locally but fails after deployment. What are the first 3 things you check?

I first find the exact failure point in the deployed environment instead of trying random fixes. The three fastest checks are startup logs, the active configuration, and the runtime/infrastructure contract.

- Read the application and platform logs from startup through the first failure. Capture the exception, its root cause, the active Spring profile, and the deployed build/version.
- Compare effective configuration, not just files: environment variables, mounted secrets, profile-specific properties, config-server values, database URL, credentials, and required feature flags.
- Verify deployment assumptions: Java version, container port and health probe, DNS/network access to dependencies, permissions on files, and database migration status.

Interview close: I would reproduce the smallest failing call with the same profile and configuration, then fix the evidence-backed cause.

3. You changed application.properties but changes are not reflecting. Why?

Usually the application is reading a higher-precedence property source or an older deployed artifact. Spring Boot merges configuration from command-line arguments, environment variables, profiles, external files, config servers, and the packaged classpath.

- Confirm which profile is active and inspect /actuator/env in a protected environment, or log a safe non-secret property at startup.
- Check environment variables such as SERVER_PORT or SPRING_DATASOURCE_URL; they can override application.properties. Also check command-line --key=value arguments and Helm/Kubernetes values.
- Confirm the correct image/JAR was rebuilt and rolled out. A packaged application.properties does not reload at runtime unless an explicit refresh/config-reload mechanism is configured.

Interview close: I avoid printing secrets, but I always make the active profile and deployed build identifier observable.

6. You need different configs for dev, test, and prod. How will you design this?

I keep shared defaults in application.yml and environment-specific overrides in application-dev.yml, application-test.yml, and application-prod.yml. The deployment selects the profile with SPRING_PROFILES_ACTIVE or a platform setting.

- Commit only safe defaults and non-secret values. Use test containers or an in-memory/local dependency for test, and production-like endpoints only in prod configuration.
- Inject secrets from a secret manager, Kubernetes Secret, vault, or CI/CD environment variables - never place production passwords in Git or profile files.
- Validate required production properties on startup with @ConfigurationProperties plus validation annotations, and fail fast if a critical setting is absent.

Interview close: The important detail is that profiles change configuration, not application behavior through scattered if/else checks.

12. You deployed a new version but users still hit the old behavior. Why?

I verify the request path end to end. This is commonly a rollout, routing, cache, or client-version issue rather than a code issue.

- Check the image digest/JAR version on every running instance and whether all pods/VMs completed the rollout. A failed readiness probe can leave old replicas serving traffic.

- Inspect load-balancer, ingress, blue-green/canary routing, sticky sessions, and DNS/CDN/API-gateway caching. Clear only the cache layer known to hold the response.
- Return a build/version header or expose it through a protected actuator info endpoint, then correlate a user request with the instance handling it.

Interview close: I do not assume deployment success means traffic has switched; I prove it using the serving instance's version.

13. How do you secure sensitive values like DB passwords in Spring Boot?

I keep secrets out of source code, properties files, logs, and container images. The app receives them at runtime from an approved secret store or deployment platform.

- Use a managed secret service or Kubernetes Secrets with workload identity/RBAC. Inject as environment variables or mounted files with least-privilege access.
- Bind secrets through `@ConfigurationProperties` without logging values. Sanitize actuator endpoints and restrict access to them.
- Rotate credentials, use short-lived credentials where supported, and test the rotation path. Repository scanning and CI checks should prevent accidental commits.

Interview close: Encryption of a secret in Git is still weaker than not shipping the secret in the artifact at all.

15. Your logs are missing in production. Where do you look?

I trace the logging path from the process to the central log viewer. Missing logs can be a logger-level setting, an appender/container stream issue, or a collector/indexing problem.

- Check the process/container stdout and stderr first, including whether the application starts and the configured root/package log levels allow the message.
- Inspect Logback/Log4j configuration, active profile, writable log path, file rotation, disk space, and permissions if files are used.
- Check the platform collector/agent, namespace and labels, ingestion filters, retention, index/query time range, and whether sampling dropped the event.

Interview close: I log a structured request ID and build version so one known test request can be located across the pipeline.

18. How do you reduce startup time of a Spring Boot application?

I measure startup first using startup metrics or `ApplicationStartup`, then remove unnecessary work from the critical path. The largest gains usually come from fewer beans, less classpath scanning, and deferring nonessential initialization.

- Enable only required starters and auto-configurations; remove unused dependencies and avoid broad component scans across unrelated packages.
- Move slow warm-ups, remote calls, and cache population out of startup or make them readiness-gated only when truly required. Do not block startup on optional integrations.
- Use production options appropriate to the platform, such as lazy initialization selectively, AOT/native images when justified, and a tuned JVM/container memory setup.

Interview close: I protect correctness: speeding startup must not make a ready instance unable to serve its required traffic.

19. You need to execute code only once after application startup. How?

For application-local initialization, I use `ApplicationRunner` or `CommandLineRunner`. Both run after the Spring context is ready; `ApplicationRunner` is usually cleaner because it receives parsed arguments.

- Use an ordered runner when multiple startup tasks have dependencies, and make the task idempotent because restarts and multiple replicas are normal.
- For a cluster-wide once-only task, do not rely on a runner alone. Use a database lock, leader election, a migration tool, or a dedicated one-off job.

- If the task is critical to serving traffic, expose readiness only after it succeeds; otherwise handle failure separately and keep startup fast.

Interview close: 'Once' must be defined: once per JVM, once per deployment, or once for the whole cluster.

23. What strategy do you use for graceful shutdown of Spring Boot apps?

I stop accepting new traffic, allow in-flight work to finish within a bounded timeout, then close resources cleanly. Spring Boot supports graceful shutdown with `server.shutdown=graceful` and `spring.lifecycle.timeout-per-shutdown-phase`.

- Make readiness fail before termination so the load balancer removes the instance. Set the platform termination grace period longer than the application shutdown timeout.
- Ensure consumers stop polling, scheduled work is interrupted safely, executors are shut down, and database/message resources close after in-flight work completes.
- Make operations idempotent because a request may be retried or the process may still be killed after the grace period.

Interview close: I validate this with a rolling deployment while sending controlled traffic and checking for dropped requests.

25. Your Spring Boot app fails only when deployed in Docker. Why?

I compare the container runtime contract with local execution: image contents, Java/runtime version, configuration, network, filesystem, and resource limits.

- Read docker logs and inspect the actual image/tag. Common issues are an incorrect ENTRYPOINT, wrong JAR path, missing layered files, or a Java version mismatch.
- Check container-only configuration: localhost points to the container itself, DNS/service names differ, ports may not be exposed, and env variables/secrets may be absent or malformed.
- Inspect cgroup CPU/memory limits, non-root file permissions, timezone/locale, certificate store, and health-check path/port.

Interview close: I reproduce with the same image and docker run environment locally, rather than running the project directly from the IDE.

28. How do you test production-like behavior locally?

I make local tests resemble the production contract: the same packaged artifact, profile, external dependency versions, security path, and resource constraints where practical.

- Run the container image rather than IDE-only execution, and use Testcontainers or Docker Compose for the same database, message broker, and cache technology.
- Use sanitized production-like data volumes, representative indexes, realistic latency/failure injection, and the same migration scripts.
- Exercise observability, authentication, timeouts, concurrency, and resource limits. A staging or ephemeral environment is still needed for a final integration check.

Interview close: The goal is not to copy production data; it is to reproduce the production interfaces and failure modes safely.

29. Your service needs zero downtime deployment. How do you achieve it?

I use rolling, blue-green, or canary deployment with health checks and backward-compatible changes. Zero downtime is a system property: application, load balancer, database, and clients must all cooperate.

- Use readiness checks to route only to fully initialized instances and graceful shutdown to drain old instances. Keep enough healthy capacity during the rollout.
- Release database changes with expand-contract migrations: add compatible schema first, deploy compatible code, migrate data, then remove old fields in a later release.
- Make APIs and messages backward compatible during mixed-version operation, watch error/latency metrics, and retain a quick rollback path.

Interview close: I explicitly test a mixed old/new fleet because that is the normal state during a rollout.

2. Performance, capacity, and resilience

Start with measurements and protect the service from slow or failing dependencies.

2. A REST API is slow only in production, not locally. How do you debug it?

I establish where the time is spent using production-safe metrics and traces. A local laptop rarely matches production data volume, network latency, contention, or infrastructure limits.

- Compare p50, p95, and p99 latency by endpoint, instance, status code, and dependency. A trace tells me whether time is in controller work, database, downstream HTTP, queueing, or serialization.
- Check production-only differences: database execution plan/indexes and connection wait time, payload size, TLS/DNS/network latency, CPU throttling, GC pauses, thread-pool queues, and cache hit rate.
- Take a focused thread dump or profiler sample only under safe load, then correlate the slow period with deployment, traffic, and infrastructure events.

Interview close: I optimize only after I have a measured bottleneck and a before/after latency metric.

4. Your Spring Boot service crashes under high traffic. What could be the reasons?

I first identify the termination reason: JVM out-of-memory, Kubernetes OOMKilled, liveness restart, process exit from an uncaught fatal error, or node/platform eviction. The fix depends on that evidence.

- Look for heap growth, allocation spikes, unbounded queues/caches, large request bodies, response buffering, thread creation, or connection leaks causing memory pressure.
- Check CPU throttling, blocked thread pools, exhausted HTTP/database pools, file descriptors, rate limits, and downstream timeouts that create a retry storm.
- Set bounded queues, sensible concurrency/timeouts, request limits, backpressure and rate limiting. Right-size JVM heap relative to the container limit and preserve heap dumps for OOM diagnosis.

Interview close: Adding replicas can hide a leak temporarily; I still find the resource that is growing without a bound.

8. A database connection pool is getting exhausted. How do you identify and solve it?

I inspect pool metrics and the database session view to learn whether connections are leaked, held too long, or simply undersized for a legitimate concurrency level.

- Monitor Hikari active, idle, pending, max, and timeout metrics; correlate them with endpoint traffic, transaction duration, and database CPU/lock waits.
- Find long-running queries and transactions. Common causes are missing indexes, N+1 queries, remote calls inside @Transactional, slow result processing, and connections not returned after manual JDBC use.
- Fix the query/transaction boundary first. Set a pool size based on database capacity, configure connection and leak-detection thresholds carefully, and limit request concurrency so the database remains stable.

Interview close: Increasing maximumPoolSize without checking database limits can turn an application issue into a database outage.

9. How do you handle API failures when calling another microservice?

Every outbound call gets an explicit timeout, a bounded retry policy only when safe, and a fallback or clear error path. I treat the remote service as unreliable by design.

- Set connect, read/response, and overall request timeouts. Use a client with connection-pool limits and propagate a correlation ID for troubleshooting.
- Retry only transient failures and idempotent operations, using exponential backoff plus jitter. Do not retry validation errors, and use idempotency keys for retried writes.

- Use circuit breakers, bulkheads, rate limits, cached/stale fallback where acceptable, and queue asynchronous work when the business flow permits it.

Interview close: The best fallback depends on the business rule: fail fast, return cached data, or accept work asynchronously - never silently return incorrect data.

10. Your application memory usage keeps increasing over time. What do you suspect?

I suspect a memory leak or an unbounded retention pattern until proven otherwise. The key question is whether used heap returns to a baseline after a full GC under stable load.

- Collect heap and GC metrics, then take two heap dumps during growth and compare retained objects. Check for static collections, caches without eviction, listeners, ThreadLocals, classloader retention, and unclosed resources.
- Also inspect non-heap memory: direct buffers, native libraries, thread stacks, metaspace, and container memory. JVM heap graphs alone do not explain every OOMKilled event.
- Bound caches and queues, close streams/clients, remove listeners, and verify the fix with a soak test. Configure heap dumps and alerts before the process is killed.

Interview close: High memory is not automatically a leak; a steadily growing retained-object set after GC is stronger evidence.

11. How do you implement retry logic for failed REST calls?

I use a retry library such as Resilience4j or Spring Retry around a named outbound client, with strict conditions. Retrying blindly is a common source of duplicate writes and cascading failures.

- Retry timeouts, connection failures, and selected 5xx/429 responses only when the operation is idempotent or protected by an idempotency key. Never retry most 4xx responses.
- Use a small maximum attempt count, exponential backoff, and jitter. The total retry budget must fit inside the caller's timeout and user latency expectation.
- Record attempts, final outcome, and latency metrics without logging secrets. Combine retries with a circuit breaker so a dead dependency is not hammered.

Interview close: For payment or create operations, I would rather use an idempotency key or async reconciliation than assume a retry is safe.

17. One microservice is down and others are failing. How do you prevent this?

I prevent failure amplification with timeouts, circuit breakers, bulkheads, and degraded behavior. A synchronous dependency should not be allowed to consume all threads while it is unavailable.

- Set short, realistic timeouts at each call boundary and avoid multiplying retries across layers. Use one owner for retries where possible.
- Partition resources: separate thread/connection pools or concurrency limits for expensive dependencies, so one outage cannot starve unrelated endpoints.
- Open a circuit breaker after a failure threshold, serve a safe fallback or fail fast, and recover through controlled half-open probes. Alert on dependency health and error budget impact.

Interview close: I also revisit whether the call must be synchronous; events or a queue can remove tight availability coupling.

20. How do you handle long-running background jobs in Spring Boot?

I do not hold an HTTP request open for a long job. I accept the request, validate it, persist/queue work, return a job ID, and expose status or a callback when it completes.

- Use a durable queue, Spring Batch, a scheduler, or a dedicated worker depending on the workload. Persist job state, progress, retry count, and failure reason.
- Make handlers idempotent, use leases/locks to avoid duplicate processing, set concurrency limits, and provide cancellation/retry behavior when the business requires it.
- For cluster deployments, avoid in-memory executors as the source of truth. Monitor queue depth, age, failure rate, throughput, and worker saturation.

Interview close: @Async is useful for small in-process tasks, but it is not a durable job system and work is lost on process failure.

21. Your API returns correct data but response time is inconsistent. Why?

I look at latency percentiles and request classes, not the average. Tail latency often comes from occasional queuing, cache misses, slow queries, GC pauses, or a noisy dependency.

- Break down p50/p95/p99 by endpoint, response size, tenant, instance, and dependency. Distributed traces reveal whether the variance is before, inside, or after the service.
- Inspect database lock waits/execution plans, connection-pool waits, thread-pool queues, cache hit rate, JVM GC pauses, CPU throttling, and network/TLS/DNS behavior.
- Fix the dominant source: index/query changes, bounded concurrency, cache tuning, reduced payloads, pool tuning, or resource allocation. Verify p95/p99 improvement under representative load.

Interview close: Correct data only proves functional correctness; service quality is measured by tail latency and reliability under load.

22. How do you trace a request across multiple microservices?

I propagate a standard trace context on every inbound and outbound call and send logs, metrics, and spans to a central observability system. In modern Spring Boot, Micrometer Tracing is the usual integration point.

- Generate or accept a trace ID at the edge, propagate W3C traceparent headers through HTTP and messages, and include traceId/spanId in structured logs.
- Create spans for meaningful boundaries such as database queries, remote calls, messaging, and asynchronous jobs. Do not create noisy spans for every tiny method.
- Use sampling appropriate to traffic and error investigation, secure telemetry so it does not contain credentials or full sensitive payloads, and verify propagation with an end-to-end test.

Interview close: A correlation ID is useful, but distributed tracing adds parent-child timing needed to locate the slow service.

26. How do you implement rate limiting for APIs?

I enforce limits at the earliest appropriate layer - usually API gateway/ingress for broad protection, and in the service for business or tenant-specific rules. A token-bucket or leaky-bucket policy is common.

- Key the limit by a trusted identity such as API key, authenticated user, tenant, or client application - not an untrusted forwarded header alone.
- Use a shared, atomic store such as Redis for multi-instance limits, or gateway-native distributed limiting. Return 429 with a Retry-After header when possible.
- Choose different limits for login, expensive endpoints, and internal traffic; monitor rejected requests and avoid allowing a limit bypass through inconsistent routes.

Interview close: Rate limiting complements authentication and capacity planning; it is not a substitute for fixing an expensive endpoint.

3. Spring behavior, transactions, and API design

These questions test whether you understand framework behavior in real deployments.

5. Multiple beans of the same type exist and Spring throws an error. How do you fix it?

Spring cannot choose a single bean for an injection point. I first decide whether one implementation is the normal default or whether the caller genuinely must choose.

- Use `@Primary` on the default implementation when there should normally be one choice. This keeps simple injection sites clean.
- Use `@Qualifier` at the injection point and bean definition when the choice is intentional, for example a read versus write client. Prefer constructor injection so the dependency is explicit.
- Inject `List<MyType>` or `Map<String, MyType>` only when the caller should run all/select dynamically. Remove accidental duplicate component scans or test configuration beans.

Interview close: I avoid resolving it with field names by accident; the selected implementation should be clear to the next developer.

7. Your API works fine but returns 401 randomly. What could cause this?

A 401 should be investigated as an authentication path inconsistency. I correlate failures by user/client, token issuer/key ID, serving instance, gateway, and time.

- Check token expiry, clock skew between services, refresh-token races, signing-key rotation/JWK cache refresh, and whether sessions are sticky/shared correctly across replicas.
- Compare gateway and service authentication behavior. A missing Authorization header after a proxy, CORS preflight mishandling, or a route bypass can appear random to users.
- Log only safe identifiers and reason codes such as expired, signature-invalid, missing-header, or audience mismatch. Never log raw tokens.

Interview close: If only a subset of instances fail, I compare their clock, configuration, key cache, and deployed version immediately.

14. What happens if `@Transactional` fails midway? How do you verify rollback?

By default, Spring rolls back a transaction when an unchecked `RuntimeException` or `Error` escapes the transactional method. Checked exceptions do not trigger rollback unless `rollbackFor` is configured. The transaction covers database work in its boundary, not arbitrary remote side effects.

- Make sure the method is called through the Spring proxy: self-invocation, private methods, or an unmanaged object can prevent `@Transactional` from applying. Also check propagation and isolation settings.
- Write an integration test that calls the method, triggers the failure after an insert/update, then starts a fresh transaction and asserts the database has no partial change.
- Do not catch and swallow the exception inside the transaction unless the transaction is explicitly marked `rollback-only`. For external calls, use an outbox/saga/compensation pattern rather than expecting database rollback to undo them.

Interview close: I verify rollback by querying committed state from a new transaction, not by inspecting an in-memory entity in the same persistence context.

16. How do you handle versioning of REST APIs without breaking clients?

I make changes additive by default and treat existing clients as a contract. Version only when an incompatible semantic or structural change is truly needed.

- Prefer adding optional fields/endpoints and keeping old fields valid. Clients should ignore unknown JSON fields; servers should tolerate absent optional fields where possible.

- For a breaking change, use a clear version boundary such as /api/v1 and /api/v2, or a vendor media type if that is the organization standard. Document lifecycle and deprecation dates.
- Keep both versions observable and tested by contract/integration tests. Track usage before retiring v1, communicate migration steps, and set a retirement policy.

Interview close: Version names alone do not ensure compatibility; behavior, error responses, pagination, and authentication are part of the contract too.

24. How do you manage schema changes without breaking production?

I version every schema change with Flyway or Liquibase and use expand-contract migrations so old and new application versions can coexist during a rollout.

- Expand first: add nullable columns, new tables/indexes, or new structures in a backward-compatible way. Test migration time and locking on production-like data.
- Deploy code that can read/write both forms, backfill asynchronously in controlled batches, validate completion, and switch reads only when data is ready.
- Contract later: remove old columns, constraints, or tables only after all old application versions and consumers are gone. Take backups and have a tested rollback/forward-fix plan.

Interview close: I never pair a destructive column rename/drop with code that requires the new schema in a single non-reversible deployment step.

27. What happens if an exception occurs inside a filter or interceptor?

It depends on where it occurs and how it is handled. An exception from a Servlet Filter may occur before the controller and may bypass normal @ControllerAdvice handling; an interceptor can be handled differently depending on the dispatcher flow.

- For security/auth filters, catch expected failures, set the correct status and response body using the security entry point or handler, and stop the chain. Do not continue after writing an error response.
- For unexpected failures, log safely with correlation IDs and route through a consistent error mechanism where possible. Avoid writing a response twice after it is committed.
- Test filter/interceptor exceptions explicitly: status, body, headers, trace ID, and whether downstream controller code was prevented from running.

Interview close: I distinguish 401 (not authenticated) from 403 (authenticated but not authorized) and keep error bodies free of internal details.

4. Practical project habits

A strong interview answer is specific about observability, trade-offs, and validation.

30. What are common mistakes freshers make in Spring Boot projects?

The pattern is usually treating a local demo as a production service. I would mention these mistakes together with the habit that prevents each one.

- Putting secrets in application.properties or source code; instead use external secret management and never log credentials/tokens.
- Using field injection, catching Exception everywhere, returning entities directly, or exposing stack traces; instead use constructor injection, focused exceptions, DTOs, and a consistent error contract.
- Ignoring validation, pagination, timeouts, connection pools, transactions, indexes, and integration tests; instead validate input, bound resources, and test real dependency behavior.
- Adding retries without idempotency, using @Async as a durable job queue, and deploying schema-breaking changes; instead design for failure, restarts, and mixed-version rollouts.

Interview close: The mature habit is to ask: what happens under bad input, partial failure, load, restart, and deployment - then make the answer observable and testable.